LAB REPORT

Lab Title	Sorting Algorithms	Lab No.	07
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/14		

Lab description/objectives:

1. Demonstrate understanding of sorting methods in C programming.
2. Implement sorting functions with customizable as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

Bubble Sort:

```
#include <stdio.h>
#define Asize 10
typedef double Elementype;
void printA(Elementype *a, int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%.1f ", a[i]);
    }
    printf("\n");
}
void reset(Elementype *a, int size){
    Elementype b[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    for (int i = 0; i < size; i++){
        a[i] = b[i];
    }
}
void swap(Elementype *a, Elementype *b)
{
    Elementype tmp = *a;
    *a = *b;
    *b = tmp;
}
void BubbleSort(Elementype *a, int size, char type)
{
    int end = size - 1;
    while (end)
    {
```

```
        for (int i = 0; i < end; i++)
        {
            if (a[i] > a[i + 1])
                swap(&a[i], &a[i + 1]);
        }
        end--;
    }
    if (type == 'd')
    {
        for (int i = 0; i < size / 2; i++)
        {
            swap(&a[i], &a[size - i - 1]);
        }
    }
}
int main()
{
    Elementype a[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    printf("The elements in the array before sorting is:\n");
    printA(a, Asize);
    BubbleSort(a, Asize, 'a');
    printf("The elements in the array after sorting in ascending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    BubbleSort(a, Asize, 'd');
    printf("The elements in the array after sorting in descending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    return 0;
}
Selection Sort:
#include <stdio.h>
#define Asize 10
typedef double Elementype;
void printA(Elementype* a, int size){
    for (int i = 0; i < size; i++){
        printf("%.1f ", a[i]);
    }
    printf("\n");
}
void swap(Elementype* a, Elementype* b){
    Elementype tmp = *a;
    *a = *b;
```

```
    *b = tmp;
}
void reset(Elementype *a, int size)
{
    Elementype b[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    for (int i = 0; i < size; i++)
    {
        a[i] = b[i];
    }
}
void SelectionSort(Elementype* a, int size, char type){
    int minidx;
    for (int i = 0; i < (size - 1); i++){
        minidx = i;
        for (int j = i + 1; j < size; j++){
            if (a[i] > a[j]){
                minidx = j;
            }
        }
        swap(&a[i], &a[minidx]);
    }
    if (type == 'd'){
        for (int i = 0; i < size / 2; i++){
            swap(&a[i], &a[size - i - 1]);
        }
    }
}
int main(){
    Elementype a[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    printf("The elements in the array before sorting is:\n");
    printA(a, Asize);
    SelectionSort(a, Asize, 'a');
    printf("The elements in the array after sorting in ascending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    SelectionSort(a, Asize, 'd');
    printf("The elements in the array after sorting in descending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    return 0;
}
```

Quick Sort:

```
#include <stdio.h>
```

```
#define Asize 10
typedef double Elementype;
void printA(Elementype *a, int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%.1f ", a[i]);
    }
    printf("\n");
}
void swap(Elementype *a, Elementype *b)
{
    Elementype tmp = *a;
    *a = *b;
    *b = tmp;
}
void reset(Elementype *a, int size)
{
    Elementype b[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    for (int i = 0; i < size; i++)
    {
        a[i] = b[i];
    }
}
int Partition(Elementype* a, int low, int high){
    Elementype pivot = a[low];
    while (low < high){
        while (low < high && a[high] >= pivot){
            high--;
        }
        a[low] = a[high];
        while (low < high && a[low] <= pivot){
            low++;
        }
        a[high] = a[low];
    }
    a[low] = pivot;
    return low;
}
void QuickSortA(Elementype* a, int low, int high){
    if (low < high)
    {
        Elementype pivot = Partition(a, low, high);
```

```
        QuickSortA(a, low, pivot - 1);
        QuickSortA(a, pivot + 1, high);
    }
}
void QuickSort(Elementype* a, int size, char type){
    QuickSortA(a, 0, size - 1);
    if (type == 'd'){
        for (int i = 0; i < size / 2; i++)
        {
            swap(&a[i], &a[size - i - 1]);
        }
    }
}
int main(){
    Elementype a[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    printf("The elements in the array before sorting is:\n");
    printA(a, Asize);
    QuickSort(a, Asize, 'a');
    printf("The elements in the array after sorting in ascending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    QuickSort(a, Asize, 'd');
    printf("The elements in the array after sorting in descending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    return 0;
}
```

Program output:

```
The elements in the array before sorting is:
4.4 3.3 2.2 5.5 1.1 6.6 7.7 10.0 9.9 8.8
The elements in the array after sorting in ascending order is:
1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9 10.0
The elements in the array after sorting in descending order is:
10.0 9.9 8.8 7.7 6.6 5.5 4.4 3.3 2.2 1.1
```

Analysis:

1. **Bubble Sort**

- Choose the appropriate loop to ensure the smallest number gradually come to the front(if ascending)
- Use another array to perform the descending order.

2. **Selection Sort**

- Understand the logic of choose the appropriate size number to its position.

3. **Quick Sort**

- Clear the logic of use the partition and recursion.

- Use another function to perform the descending order.

Conclusion：

When using the Bubble sort, we need to understand its logic that to continuously compare the neighbor two elements until the smallest element come to the front. When using the Selection sort, we need to understand its logic that set the *n*th smallest number on the *n*th position. When using the Quick sort, we need to understand how to choose the pivot, how to partition the array and how to use the recursion.